Copy page

Course Project Guidelines

The course project is a central component of this course, accounting for **50%** of your final grade. It is a substantial, collaborative undertaking designed for teams of **2 to 4 students** to apply and expand upon concepts from lectures by building and deploying a **stateful cloud-native application**. This project emphasizes cloud and edge computing technologies covered in class, including containerization, orchestration, persistent storage, and monitoring.

Project Deliverables · Project Idea · Project Proposal · Presentation · Final Project Deliverable · Tips

Project Deliverables

The course project consists of three major deliverables:

1. **Project Proposal** (due **Sunday, March 1, 2026, 11:59 PM**): **15% of final grade**
   - A Markdown document outlining the project's motivation, objectives, features, and tentative plan.
2. **Presentation** (delivered during Lectures 11 and 12, **Wednesday, March 25 & April 1, 2026**): **10% of final grade**
   - A 6-minute in-class presentation showcasing your project's features and technical implementation as completed by **March 24, 2026**. While project development may continue until the Final Project Deliverable deadline, the presentation must reflect only the work completed by March 24 to ensure fairness across presentation sessions.
   - The presentation will be graded 5% by peer review and 5% by instructor & TAs using the same rubric.
3. **Final Project Deliverable** (due **Saturday, April 4, 2026, 11:59 PM**): **25% of final grade**
   - **Source Code**: Delivered as a public or private Git repository on GitHub. Ensure that your repository is well-organized, with clear structure and comments.
   - **Final Report**: A detailed report to ensure full reproducibility of your work, delivered as a `README.md` file in your GitHub repository.
   - **AI Interaction Record**: Documentation of 1–3 meaningful AI interactions that meaningfully influenced your project, provided as an `ai-session.md` file in the GitHub repository.
   - **Video Demo**: A video demonstration of your project, lasting between 1 and 5 minutes. The demo should highlight the key features and functionality of your application. Include the video URL in the `## Video Demo` section of your final report (the `README.md` file).

Project Idea

The course project represents a more innovative and time-consuming piece of work than the course assignments, requiring teamwork to build a **stateful cloud-native application** (e.g., a web app or service that maintains user data across sessions/restarts). It consists of four key stages: **team formation**, **project proposal**, **in-class presentation**, and **final deliverable submission by the specified deadline**.

Your project **MUST** be deployed to a cloud or edge provider and **MUST** incorporate the following technologies and features:

Core Technical Requirements

1 Containerization and Local Development (Required for ALL projects)

- Use **Docker** to containerize the application (e.g., Node.js backend, database).
- Use **Docker Compose** for multi-container setup (e.g., app + database).

2 State Management (Required for ALL projects)

- Use **PostgreSQL** for relational data persistence.

- Implement **persistent storage** (e.g., DigitalOcean Volumes ↗ or Fly Volumes ↗) to ensure state survives container restarts or redeployments.

3 Deployment Provider (Required for ALL projects)

- Deploy to either **DigitalOcean** (IaaS focus) or **Fly.io** (edge/PaaS focus).

4 Orchestration Approach (Choose ONE)

Option A: Docker Swarm Mode

- Use Docker Swarm mode for clustering and orchestration.
- Implement service replication and load balancing.

Option B: Kubernetes

- Use Kubernetes for orchestration (start with minikube locally, then deploy to a cloud-managed cluster, e.g., DigitalOcean Kubernetes ↗).
- Include Deployments, Services, and PersistentVolume for stateful data.

> **Note on Orchestration on Fly.io**
>
> Using Fly.io's built-in scaling, replication, or global distribution features alone does **not** satisfy the orchestration requirement, because these are provider-level capabilities rather than an implementation of Docker Swarm or Kubernetes.
>
> Fly Kubernetes Service (FKS) ↗ is **not** required. It is a paid beta feature ($75/month per cluster) and may have limited documentation or bugs. You may use it if you wish, but it is not recommended.
>
> For Kubernetes, please use minikube locally plus a managed Kubernetes service — DigitalOcean Kubernetes — or Docker Swarm on Fly.io/DigitalOcean. These options fully satisfy the course requirements and avoid unnecessary cost or complexity.

5 Monitoring and Observability (Required for ALL projects)

- Integrate monitoring using provider tools (e.g., DigitalOcean metrics/alerts for CPU, memory, disk; Fly.io logs/metrics).
- Set up basic alerts or dashboards for key metrics.
- *(Optional)* You may also integrate metrics/logs/traces into your frontend if you have one to make the demo clearer.

> **Note on Frontend (UI)**
>
> A frontend is **not required**, but you may include a simple web interface (e.g., built with Next.js ↗ or another framework) to make your demo and presentation clearer. The grading focus is on your backend, architecture, and deployment, not UI complexity.

Advanced Features (Must implement at least two)

- Serverless integration (e.g., DigitalOcean Functions or Fly.io for event-driven tasks like notifications).
- Real-time functionality (e.g., WebSockets for live updates).
- Auto-scaling and high availability (e.g., configure Swarm/K8s to scale based on load).
- Security enhancements (e.g., authentication/authorization, HTTPS, secrets management).
- CI/CD pipeline (e.g., GitHub Actions for automated builds/deployments).
- Backup and recovery (e.g., automated database backups to cloud storage).
- Integration with external services (e.g., email notifications via SendGrid).
- Edge-specific optimizations (e.g., global distribution on Fly.io with region-based routing).

Note on Project Requirements and Scope

> ❗ **Meeting the Core Technical Requirements is mandatory — no exceptions.** Projects must use specified tools (e.g., DigitalOcean/Fly.io, PostgreSQL) to align with course content, ensure fair grading and peer review, and keep costs/complexity manageable. Alternative platforms (e.g., AWS/GCP) are not permitted. However, students are encouraged to explore advanced features with any modern tools as long as they are compatible with the core technical requirements.

Your project **MUST** implement all core technologies and at least two advanced features. To ensure an achievable project within the ~2-month timeline, avoid these pitfalls:

- **Too simple**: A basic CRUD app without orchestration or monitoring shows insufficient mastery.
- **Too complex**: Distributed systems or ML integrations are hard to complete effectively.
- **Too broad**: A full-scale platform (e.g., a cloud-native social media app) risks shallow implementation.

The **goal** is to develop a **well-scoped, thoughtfully implemented application** that effectively showcases your understanding of cloud computing principles using the required technologies. A successful project should have a **clear purpose, achievable scope, and polished implementation of core features**.

Remember: A well-executed, focused project is far more valuable than an overly ambitious one that cannot be completed properly within the course timeline. While creativity and innovation are encouraged, please keep in mind that **this is a course project with specific learning objectives to achieve.**

Example Project Ideas

Below are some inspiring project ideas to guide your team. Smaller teams (2 members) may implement a subset of features, while larger teams (3-4 members) should aim for more features or added complexity. **All projects must fulfill the Core Technical Requirements**. The listed features illustrate potential scope rather than strict expectations. Teams are also encouraged to propose unique project ideas tailored to their interests and expertise, provided the scope is realistic and achievable within the project timeline.

1. **Collaborative Task Management Platform**

   A cloud-native web application for teams to manage tasks with real-time updates and persistent data storage.

   Key Features:

   - User authentication and team-based project management
   - Role-based access control (e.g., Admin, Member)
   - Task creation, assignment, and status tracking (e.g., To-Do, In Progress, Done)
   - Real-time task updates using WebSockets
   - Persistent storage for tasks and user data using PostgreSQL and DigitalOcean Volumes
   - Dockerized Node.js backend with Docker Compose for local development
   - Deployment on DigitalOcean with Docker Swarm for orchestration
   - Monitoring dashboard for CPU, memory, and task activity metrics
   - Automated database backups to cloud storage
   - CI/CD pipeline with GitHub Actions for automated deployments
   - Search functionality for tasks by keyword or assignee

---

2. **Event Logging and Analytics System**

   A platform to collect, store, and analyze event data (e.g., IoT sensor readings or user interactions) with real-time insights.

   Key Features:

   - User authentication and organization-based access
   - REST API for event ingestion and data retrieval

- Real-time event visualization with dynamic charts
- PostgreSQL for event storage with Fly.io persistent volumes
- Dockerized Python backend with Docker Compose for local setup
- Deployment on Fly.io with Kubernetes for orchestration
- Monitoring alerts for high event volume or system health
- Automated backups of event data to cloud storage
- Auto-scaling based on event ingestion load
- Search and filter events by timestamp, type, or source
- Serverless notifications for critical events (e.g., via DigitalOcean Functions)

---

3. **Content Sharing and Collaboration Platform**

   A cloud-based system for users to upload, share, and collaborate on content (e.g., documents, images).

   Key Features:

   - User authentication with role-based permissions (e.g., Owner, Collaborator, Viewer)
   - File upload and management with version history
   - Collaborative commenting and tagging on content
   - PostgreSQL for metadata and user data with DigitalOcean Volumes
   - Dockerized Node.js backend with Docker Compose for local testing
   - Deployment on DigitalOcean with Docker Swarm for load balancing
   - Monitoring for storage usage and API performance metrics
   - CI/CD pipeline using GitHub Actions for automated builds
   - Search functionality for content by tags or metadata
   - HTTPS and authentication for secure access
   - Export content metadata in JSON or CSV formats

---

4. **Inventory Management System**

   A cloud-native application for tracking and managing inventory with real-time updates and edge optimizations.

   Key Features:

   - User authentication with role-based access (e.g., Manager, Staff)
   - Inventory CRUD operations (create, read, update, delete) for items
   - Real-time stock updates using WebSockets
   - PostgreSQL for inventory data with Fly.io persistent volumes
   - Dockerized Python backend with Docker Compose for local development
   - Deployment on Fly.io with Kubernetes for orchestration
   - Monitoring dashboard for inventory levels and system health
   - Automated backups of inventory data to cloud storage
   - Edge routing for low-latency access across regions
   - Search and filter inventory by category, quantity, or location
   - Serverless email notifications for low-stock alerts

---

5. **Scientific Data Repository**

A platform for researchers to store, share, and analyze scientific datasets with access control and visualization.

Key Features:

- User authentication and role-based permissions (e.g., Researcher, Admin)
- Dataset upload and metadata management (e.g., tags, descriptions)
- Access control for public/private datasets
- PostgreSQL for dataset metadata with DigitalOcean Volumes
- Dockerized Python backend with Docker Compose for local setup
- Deployment on DigitalOcean with Docker Swarm for orchestration
- Monitoring for storage usage and dataset access metrics
- Visualization dashboard for dataset trends
- Serverless notifications for dataset updates
- Search functionality for datasets by metadata or tags
- Backup and recovery system for dataset integrity

Clarification on Team Size and Grading

Grading is consistent for all teams (2-4 students), with expectations scaled by team size. All teams must implement core technical requirements (Docker, PostgreSQL, storage, orchestration, monitoring) and at least two advanced features. Smaller teams (2 members) may use simpler implementations, while larger teams (3-4 members) should add complexity, as shown in Example Project Ideas.

Rubrics for the Project Proposal, Presentation, and Final Deliverable focus on quality and requirements, not team size. The Project Completion rubric adjusts code expectations: 1000+ lines per member (2-member teams), 850+ (3-member teams), 700+ (4-member teams). Individual contributions are verified via GitHub commits to ensure fairness. More members mean more coordination, which balances workload across team sizes.

Project Proposal

The project proposal should be submitted as a single file in the form of a Markdown document (with the `.md` suffix in the filename), with a maximum length of **2000 words**. Other formats (such as Microsoft Word or Adobe PDF) will not be accepted, as Markdown is the industry standard for technical documentation in software development. The project proposal should include the following five sections of the project, described clearly and concisely:

Required Sections

1. Motivation

- Identify the problem or need your project addresses
- Explain why the project is worth pursuing
- Describe the target users
- Optional: Discuss existing solutions and their limitations

2. Objective and Key Features

- Provide a clear statement of project objectives
- Describe all core features, including:
  - Chosen orchestration approach (Swarm or Kubernetes)
  - Database schema and persistent storage
  - Deployment provider (DigitalOcean or Fly.io)
  - Monitoring setup

    - Planned advanced features (at least two)
- Explain how these features fulfill the course project requirements
- Discuss the project's scope and feasibility within the timeframe

3. Tentative Plan

- Describe how your team plans to achieve the project objectives over the next few weeks
- Provide a clear breakdown of responsibilities for each team member
- Outline the plan week-by-week, but you do not need to provide exact milestone dates as the duration of the project is short

4. Initial Independent Reasoning (Before Using AI)

- Record your team's original thinking and plans **before consulting any AI tools**.
- Respond to the following prompts clearly and concisely:

    1. **Architecture choices**: Describe your early decisions for provider, orchestration, and persistent storage, and explain why you made them.
    2. **Anticipated challenges**: Identify the aspects of the project your team expected to be most difficult, and explain your reasoning.
    3. **Early development approach**: Outline your initial strategy for implementing the project and how responsibilities were intended to be divided among team members. Emphasize your rationale rather than a week-by-week plan.

This section should reflect **your team's thinking at the start of the project**, not a retrospective justification. Clarity and authenticity matter more than completeness or technical sophistication.

5. AI Assistance Disclosure

- Provide a brief reflection on how AI tools contributed to your proposal, if at all
- Answer the following prompts clearly and concisely:

    1. Which parts of the proposal were developed **without** AI assistance?
    2. If AI was used, what specific tasks or drafts did it help with?
    3. For one idea where AI input influenced your proposal, briefly explain:

        - what the AI suggested, and
        - what additional considerations, constraints, or tradeoffs your team discussed when deciding whether or how to adopt that suggestion.

Marking Rubrics

Motivation: 28% (out of 10 Points)

- **10 Points**: The motivation is sufficiently convincing, with a clear problem statement and well-defined target users. The proposal demonstrates thoughtful consideration of why this project is worth pursuing and how it benefits its intended users.
- **6 Points**: The motivation is present but lacks conviction or clarity. The problem statement or target users are vaguely defined, making it difficult to understand the project's value.
- **0 Point**: The motivation section is missing or completely irrelevant to the project scope.

Objective and Key Features: 48% (out of 10 Points)

- **10 Points**: The project objectives are clearly and precisely defined. All core technical requirements are explicitly addressed. At least two advanced features are clearly specified. The proposed scope is realistic for the team size and timeline, demonstrating strong technical understanding and thoughtful feasibility analysis.
- **7 Points**: The project objectives and key features are mostly clear and cover all required components. Minor gaps, ambiguities, or over/under-scoping are present, but the overall plan is technically sound and feasible. Advanced features are identified but may lack some implementation detail or justification.

- **4 Points**: The objectives and features are present but lack clarity, completeness, or technical depth. One or more required components are insufficiently explained, weakly justified, or appear unrealistic for the timeline. The feasibility of the proposed scope is uncertain.
- **0 Point**: The objectives and features section is missing, fundamentally unclear, or fails to address the basic course project requirements.

Tentative Plan: 14% (out of 10 Points)

- **10 Points**: The plan is clear, realistic, and well structured. Team responsibilities are articulated, and a casual reader can reasonably believe the group can complete the project on time.
- **6 Points**: The plan is present but lacks clarity, structure, or feasibility. A casual reader may not be convinced the project can be completed.
- **0 Point**: The proposed plan is missing or incomprehensible.

Initial Independent Reasoning (Before Using AI): 5% (out of 10 Points)

- **10 Points**: Provides a concise, meaningful summary of the team's initial thinking before any AI assistance. Architecture decisions, expected challenges, and early plans are clearly stated and consistent with the proposal.
- **6 Points**: Included but shallow, overly vague, or inconsistent with the rest of the proposal; shows limited evidence of genuine pre-AI reasoning.
- **0 Point**: Missing, irrelevant, or evidently written after AI usage (retroactive reasoning).

AI Assistance Disclosure: 5% (out of 10 Points)

- **10 Points**: Clear, specific reflection that identifies where AI was or was not used, how AI contributed, and what human reasoning or tradeoff analysis complemented AI output.
- **6 Points**: Reflection is present but lacks specificity; overly generic; does not clearly distinguish human vs. AI contributions.
- **0 Point**: Missing, vague, or evidently AI-generated without genuine reflection.

Submission

Submit a single Markdown document to the assignment labeled **Project Proposal** in the Quercus course website ↗ by **Sunday, March 1, 2026, 11:59 PM**. Each member of the team should make their own submission, but obviously all members in the same team should submit the same document.

> ⓘ **Note on Project Changes Post-Proposal**
>
> You are allowed to modify your project idea, features, or scope after submitting the proposal (e.g., based on challenges, or new insights). We will not enforce consistency between the proposal and your final deliverables — each will be graded independently using the provided marking rubrics.

Presentation

Each team will deliver a **6-minute presentation** during Lecture 11 (March 25, 2026) or Lecture 12 (April 1, 2026) to showcase the project's features and technical implementation as completed by **March 24, 2026**.

Presentation slots will be randomized and announced by Monday, March 2, 2026.

Teams may designate one or two members to present. However, **all team members must attend both sessions** to provide peer feedback. Exceptions will be granted only to part-time MEng students with unavoidable work conflicts.

> ⓘ For guidance on how to structure your presentation, design slides, manage timing, and deliver the presentation, see Project Presentation Guidelines.

Submission

- Submit a **written project introduction (70–100 words)** individually via Quercus ↗ by **Monday, March 23, 2026, 11:59 PM EST**.

- This introduction is used to set up peer review forms and to publish a preview of projects on the course website.
- During the presentation, teams are still expected to introduce the project themselves as part of the 6-minute time slot.

- Submit **presentation slides** individually via Quercus ↗ by **Tuesday, March 24, 2026, 11:59 PM EST**.
  - All team members must upload the same file to ensure fairness.
  - During the presentation, teams are expected to use the same slides submitted to Quercus. Minor revisions such as typo fixes are acceptable, but no substantive content changes are allowed after March 24, 2026.

Content Requirements

- **Core Requirements (Mandatory)**: Demonstrate all core technical requirements. This may be done in either a cloud environment or a local environment, depending on your deployment status by March 24, 2026.

  The core requirements including:

  - **Docker** and **Docker Compose** for containerized app setup.
  - **PostgreSQL** with **persistent storage**.
  - Orchestration with **Docker Swarm** or **Kubernetes** (e.g., replication, load balancing).
  - **Monitoring** with provider tools (e.g., DigitalOcean/Fly.io metrics, alerts).

  A live demo is strongly encouraged.
  Short recorded clips may be used as a backup for network-dependent components (see Presentation Logistics).

- **Advanced Features**: Present at least two advanced features.
  - If an advanced feature is already implemented: Briefly demonstrate it (live or via screenshot/recording) and clearly explain its purpose, design, and how it integrates with the rest of the system.
  - If an advanced feature is not yet implemented: Clearly explain the feature and its design, support the explanation with a simple mockup, diagram, example, or screenshot, and provide a clear, feasible milestone plan showing how the feature will be completed by April 4, 2026.
- Clearly highlight the application's **stateful design**, key features, and overall integration.

Expectations

- **Scope and Fairness**: The presentation must showcase **only the work completed by March 24, 2026**.

  Development may continue after this date, but any new features or major changes completed after March 24 must **not** be included in the presentation.

- **Core Requirements**: By March 24, 2026
  - If your project is already deployed: You may present and demo directly in the cloud environment (DigitalOcean or Fly.io).
  - If your project is not yet deployed: You must demonstrate all core requirements fully in a local environment (e.g., Docker Compose, Docker Swarm locally, or Kubernetes via minikube), and clearly state:
    - Which deployment provider you will use (DigitalOcean or Fly.io)
    - How the local setup maps to the planned cloud deployment

  Deployment itself is not required by March 24, but all core technical requirements must be functional and demo-ready in at least one environment.

- **Advanced Features**: May still be in progress at the time of presentation, but teams must provide a clear, realistic explanation and a concrete completion plan.
- **Demo**: Use a live or recorded demo to showcase all core features. Optionally include configuration snippets (e.g., Dockerfiles, Kubernetes YAML) for clarity.
- **Communication**: Presentations should be delivered without reading from a full script. Clear and direct oral communication is part of the evaluation. Detailed expectations are outlined in the Project Presentation Guidelines

Grading

- Worth 10% of final grade (5% peer, 5% instructor & TAs), evaluated using the Presentation Rubric.
- Peer scores are normalized across both presentation days for fairness.

Final Project Deliverable

The final project deliverable should be submitted as a URL to a public or private GitHub repository. If your repository is private, add the instructor and TAs (GitHub usernames: `cying17`, `yuel5304`, and `YirenZzz`) as collaborators so that it can be read.

Your repository must contain:

1. A final report (`README.md`)
2. Complete source code
3. An AI interaction record (`ai-session.md`)
4. A video demo

Final Report

A `README.md` file serve as the final report, in the form of a Markdown document ↗ with a maximum length of **5000 words** [1] [2]. If you include images (e.g., screenshots), ensure they are visible when the instructor or TAs view your GitHub repository in a web browser.

The report should clearly and concisely cover the following aspects:

- **Team Information**: List the names, student numbers, and preferred email addresses of all team members. Make sure these email addresses are active as they may be used for clarification requests.
- **Motivation**: Explain why your team chose this project, the problem it addresses, and its significance.
- **Objectives**: State the project objectives and what your team aimed to achieve through the implementation.
- **Technical Stack**: Describe the technologies used, including the chosen orchestration approach (Swarm or Kubernetes) and other key tools.
- **Features**: Outline the main features of your application and explain how they fulfill the course project requirements and achieve your objectives.
- **User Guide**: Provide clear instructions for using each main feature, supported with screenshots where appropriate.
- **Development Guide**: Include steps to set up the development environment, covering environment configuration, database, storage, and local testing.
- **Deployment Information**: Provide the live URL of your application.
- **AI Assistance & Verification (Summary)**: If AI tools contributed to your project, provide a concise, high-level summary demonstrating that your team:

  - Understands where and why AI was used
  - Can evaluate AI output critically
  - Verified correctness through technical means

  Specifically, briefly address:

  - Where AI meaningfully contributed (e.g.,architecture exploration, Docker/K8s configuration, debugging, documentation)
  - One representative mistake or limitation in AI output (details should be shown in `ai-session.md`)
  - How correctness was verified (e.g., testing, logs, monitoring metrics, manual inspection)

  Do **not** repeat full AI prompts or responses here. Instead, reference your `ai-session.md` file for concrete examples.
- **Individual Contributions**: Describe the specific contributions of each team member, aligning with Git commit history.
- **Lessons Learned and Concluding Remarks**: Share insights gained during development and any final reflections on the project experience.

Source Code

Your GitHub repository must contain all code required to build and run the project, organized in a clear, logical directory structure. Required components include:

- Application code (e.g., Node.js, Python)
- Dockerfiles and Docker Compose/Kubernetes configs
- Database schemas and migrations
- Monitoring setup scripts
- Environment configuration templates

Include detailed setup and runtime instructions in the `README.md`'s Development Guide for local execution.

Your code should follow consistent formatting and include appropriate comments for complex logic. Consider including a `.gitignore` file to exclude unnecessary files and dependencies from version control.

> ⓘ If your project requires sensitive credentials (e.g., API keys, database credentials) for execution, submit them in a password-protected `.zip` or `.tar.gz` file via email to TA Yiren Zhao: yiren.zhao@mail.utoronto.ca
>
> Send the password in a separate email to the TA. Both emails must be sent **by the final deliverable deadline**. Each team only needs to complete this step once.
>
> In your Development Guide section of your final report (the `README.md` file), clearly state "Credentials sent to TA".

AI Interaction Record

Your repository must include a file named `ai-session.md` to provides concrete evidence supporting the AI Assistance & Verification section in `README.md`.

The `ai-session.md` file documents **1–3 representative AI interactions** that meaningfully influenced the project. These interactions should illustrate:

- Responsible and transparent use of AI tools
- Your team's ability to critically evaluate AI output
- How AI-generated suggestions were validated, adapted, or corrected in the context of your application

For each interaction, include:

ai-session.md

```md
## Session Title (e.g., Debugging Docker Compose networking)

### Prompt (you sent to AI)

<copy/paste>

### AI Response (trimmed if long)

<copy/paste or summary>

### What Your Team Did With It

1-3 bullet points describing:

- What was useful
- What was incorrect, misleading, or not applicable to your project
- How your team verified, modified, or replaced the suggestion
```

There is no word limit for this file.

You do not need to include every AI interaction. Choose examples that best demonstrate judgment, correction, and verification.

Video Demo

Include a 1–5 minute video demo, showcasing:

- Key features in action
- User flow through the app
- Technical highlights (e.g., Docker containers, PostgreSQL queries, orchestration, monitoring)
- The app running in the deployed cloud/edge environment (e.g., via a live URL on DigitalOcean or Fly.io)

The video's URL must be included in the `## Video Demo` section of the final report (the `README.md` file). Host the video on a platform like YouTube, Dropbox, or Google Drive, ensuring access for the instructor and TAs. If under 100 MB, the video may be included directly in the GitHub repository.

Marking Rubrics

Technical Implementation: 30% (out of 20 Points)

To be evaluated by reading the final report (`README.md`), reviewing the source code, and testing application functionality.

- **20 Points**: Complete and correct implementation of all required technologies (Docker, PostgreSQL, storage, Swarm/K8s, monitoring).
- **15 Points**: Implementation is largely correct, with minor issues or weaknesses in one or two areas.
- **10 Points**: Basic implementation is present, but multiple components have noticeable issues or limitations.
- **5 Points**: Major issues exist in the implementation, significantly affecting correctness or functionality.
- **0 Points**: Missing critical technical components.

Project Completion: 30% (out of 20 Points)

To be evaluated by reading the final report (`README.md`), reviewing source code, and watching the video demo.

- **20 Points**: All proposed features work correctly with clear user flow. Meets minimum lines of meaningful code (excluding comments, `node_modules` , generated files, etc.) per member: 1000+ (2 members), 850+ (3 members), or 700+ (4 members).
- **15 Points**: Most features work as intended. Lines of meaningful code per member: 700–1000 (2 members), 600–850 (3 members), or 500–700 (4 members).
- **10 Points**: Basic features are implemented. Lines of meaningful code per member: 400–700 (2 members), 300–600 (3 members), or 250–500 (4 members).
- **5 Points**: Features are significantly unfinished, or lines of meaningful code fall below the minimum threshold for the team size.
- **0 Points**: Minimal working functionality or insufficient code contribution.

> ⓘ Lines of code (LOC) are counted using cloc ↗, including application code (e.g., JavaScript, Python, PHP), SQL, Dockerfiles, YAML, and JSON. Specify your programming language in the final report's Features section. Comments, generated files (e.g., `node_modules`), and non-source files (e.g., `.gitignore` , Markdown) are excluded. Functionality is prioritized over raw LOC.

Documentation and Code Quality: 15% (out of 20 Points)

To be evaluated by reading the final report (`README.md`) and reviewing code organization.

- **20 Points**: Comprehensive and well-structured `README.md` covering the user guide, development setup, and deployment information, with clear instructions, a well-organized codebase, consistent coding style, and regular, meaningful Git commits.
- **15 Points**: Good documentation overall, with minor gaps in clarity, structure, or consistency.
- **10 Points**: Basic documentation is present but lacks essential details, or the codebase shows inconsistent organization or style.
- **5 Points**: Documentation is incomplete or unclear, or the codebase is poorly organized or difficult to follow.
- **0 Points**: Documentation is missing or the codebase is chaotic and unreadable.

AI Reflection Quality: 5% (out of 20 Points)

- **20 Points**: Clear, specific, and technically grounded explanation of where AI contributed, including at least one incorrect or suboptimal AI output, how it was identified, and how correctness was verified (e.g., testing, logs, monitoring). Demonstrates strong understanding of the system's architecture and implementation.
- **10 Points**: Reflection is present but lacks technical depth or specificity. AI contributions are described at a high level, with limited discussion of errors, limitations, or verification. Evidence of human judgment and validation is present but incomplete or weak.
- **0 Point**: Reflection is missing, superficial, or reads as generic or AI-generated without evidence of genuine understanding, critical evaluation, or technical verification.

Individual Contributions: 20% (out of 20 Points)

Individual marks will be assigned to each team member based on reading the final report (`README.md`) and reading the commit messages in the commit history in the GitHub repository.

- **20 Points**: The team member has made a fair amount of contributions to the project, without these contributions the project cannot be successfully completed on time.
- **15 Points**: The team member has made less than a fair amount of contributions to the project, *or* without these contributions the project can still be successfully completed on time.
- **5 Points**: The team member has made less than a fair amount of contributions to the project, *and* without these contributions the project can still be successfully completed on time.
- **0 Point**: The team member has not made any contributions to the project.

Bonus Points

Each of the following achievements grants 2% bonus points to the final project deliverable grade:

- **Notable Innovation in Implementation**: Implement a highly creative feature that exceeds course project requirements by using course technologies in a novel way (e.g., predictive alerts using monitoring tools or innovative edge routing on Fly.io for low-latency data delivery).
- **Exceptional User Experience**: Deliver a polished application interface or user flow beyond basic requirements, such as advanced visualizations (e.g., real-time dashboards), optimized performance (e.g., low-latency edge delivery), or comprehensive error handling and user feedback.
- **High-Quality Open Source Project**: Create a project usable by external developers, including:
  - Clear contribution guidelines (e.g., how to submit issues or pull requests)
  - Well-documented APIs or components (e.g., REST endpoints, Docker/Kubernetes configs)
  - Detailed installation and setup instructions for local and cloud deployment
  - An MIT or similar open-source license
  - A professional README with sections for installation, usage, and contribution guidelines

Note: Bonus points are awarded for exceeding course expectations, with a maximum of 6% extra credit added to the final project deliverable grade (25% of final grade). Innovative use of cloud/edge technologies or robust monitoring is highly valued.

Submission

Submit a single URL — the URL to your team's GitHub repository — to the assignment labeled **Final Project Deliverable** in the Quercus course website ↗. Each member of the team should make their own submission, but obviously all members in the same team should submit the same URL.

⚠ The deadline for the course project is **Saturday, April 4, 2026**, at **11:59 PM** Eastern time, and late submissions will **NOT** be accepted. Do remember to add the instructor and TAs (GitHub usernames: `cying17`, `yuel5304`, and `YirenZzz`) as collaborators to the GitHub repository, if it is private, **before** the deadline.

ⓘ **Deployment Availability During Grading**

Your deployment should remain online during the grading period. We will post an announcement once grading is complete, and after that you are free to shut it down.

Occasional downtime caused by platform instability or external attacks will **not** affect your grade, as this introductory course does not cover production-scale reliability or security. If we cannot access your deployment during grading, we will contact your team and give you time to restart it.

Tips and Suggestions

Using GitHub for Project Management and Effective Collaboration

Effective teamwork depends on efficient communication and collaboration, often more so than completing individual tasks. To streamline your team's workflow, I highly recommend using **GitHub** as your central collaboration hub. Treat your GitHub repository as your team's central workspace — it's not just for code storage but also a powerful tool for organizing and managing your entire project. Here's how to leverage its full functionality:

- **Commit Frequently and Meaningfully**: Every change to the code should be committed to the repository immediately, accompanied by a **complete, concise, and meaningful commit message**. This helps everyone stay on the same page and track progress over time.
- **Use Branches and Pull Requests**: Create a new branch for each feature or task. Once the feature is complete, submit a pull request for review. This ensures code quality, facilitates collaboration, and makes it easier to integrate changes.
- **Use GitHub Issues for Task Management**: Create GitHub Issues to maintain a "to-do" list of outstanding tasks. Assign issues to team members, add labels for priority or category, and close them once completed. This keeps your workflow organized and transparent.
- **Leverage GitHub Discussions for Communication**: Use GitHub Discussions to document all team conversations, decisions, and brainstorming sessions. This ensures that nothing gets lost, and you can always refer back to previous discussions if needed.
- **Document Everything in the GitHub Wiki**: Use the GitHub Wiki as your project journal to record major decisions, challenges, and solutions. This creates a valuable reference for your team and anyone who might review your project in the future.

Check this page for a step-by-step guide on using GitHub to manage tasks and collaborate effectively.

---

1. There is no minimum length requirement for the final report in `README.md`. Other formats (such as Microsoft Word or Adobe PDF) will not be accepted, as Markdown is the standard documentation format in software development. ↩
2. You may reuse relevant content from your project proposal where appropriate. ↩

Last updated on January 5, 2026

MIT 2026 © Nextra.